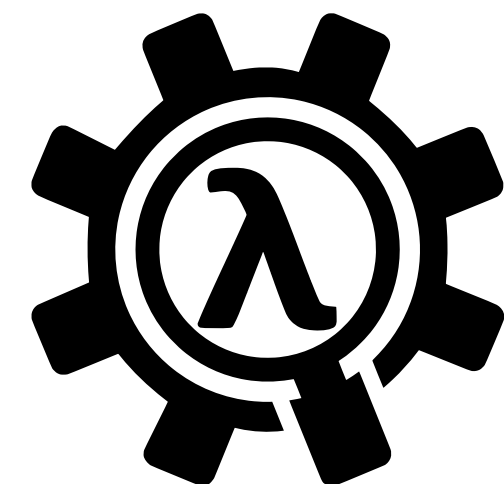


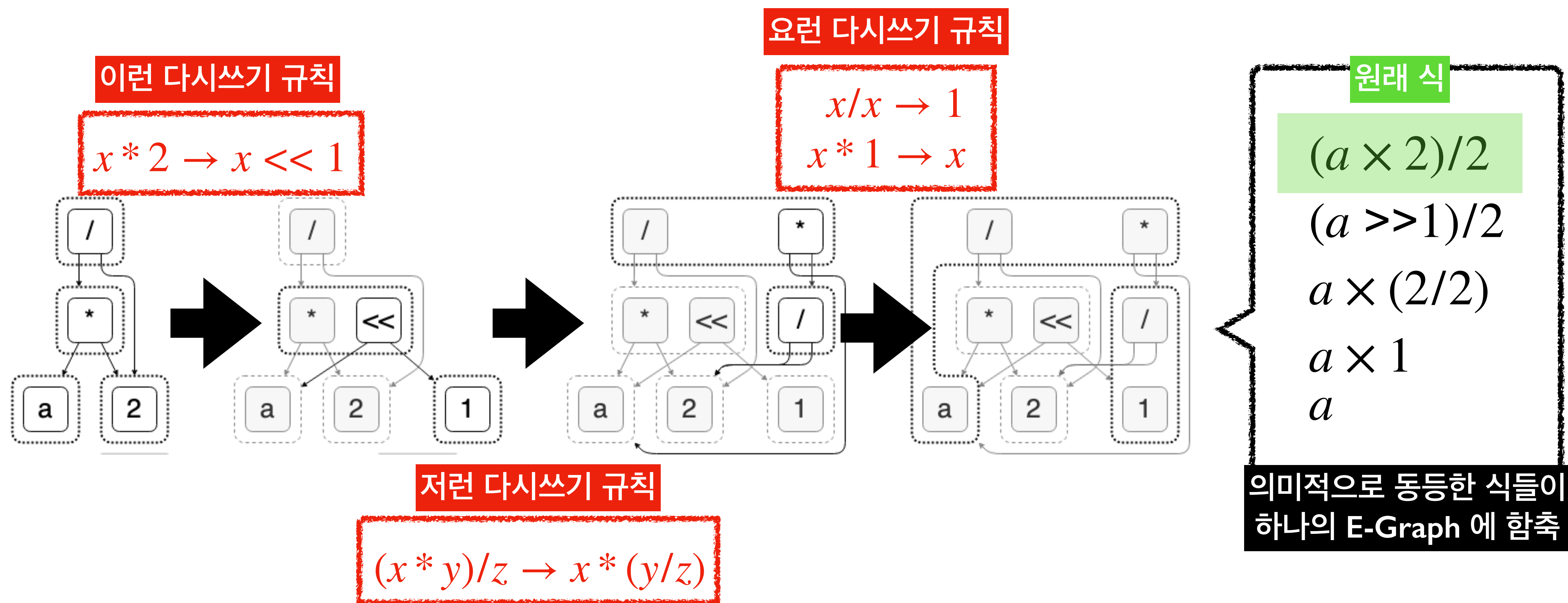
컴파일러 구현체로부터 최적화 규칙을 에이전트 기반으로 추출하기

**Agent-Based Extraction of Equality Saturation Rules
from Compiler Implementations**

한양대학교 프로그래밍 시스템 연구실 / 석사 과정 장준민



의미가 동등한 식 모두 모으고 가장 좋은 거 뽑기



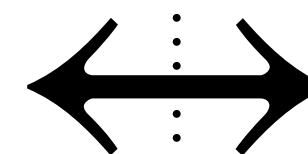
- 다시쓰기 규칙을 적용하면 적용할 수록, 동등한 식들이 **E-Graph** 에 모인다
- 모두 모이면 그 중에서 가장 좋은 프로그램 (식) 을 뽑으면 된다

기존 최적화기의 규칙을 다시쓰기 규칙으로 옮기기란

기존 최적화기 규칙 끝 (절차적)

```
if 연산자가 곱셈
  if 오른쪽 피연산자가 상수
    if 그 상수가 2의 거듭제곱
      시프트량 ←  $\log_2(\text{그 상수})$ 
      결과 ← (왼쪽 피연산자 << 시프트량)
      결과 반환
```

동등식 모으기 엔진을 위한 규칙 끝 (선언적)

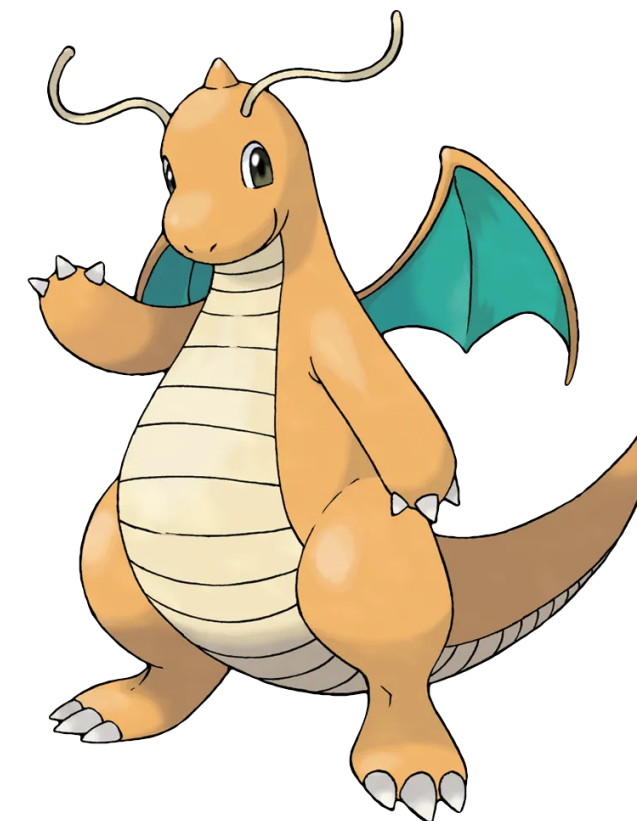
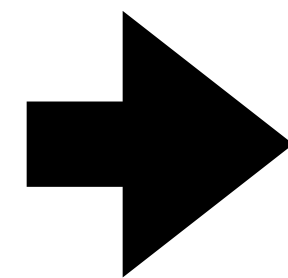


$$x * 2^k \rightarrow x \ll k$$

이건 무엇을 어떤걸로 바꾸는 지 눈에 잘 들어오는 일부 쉬운 예

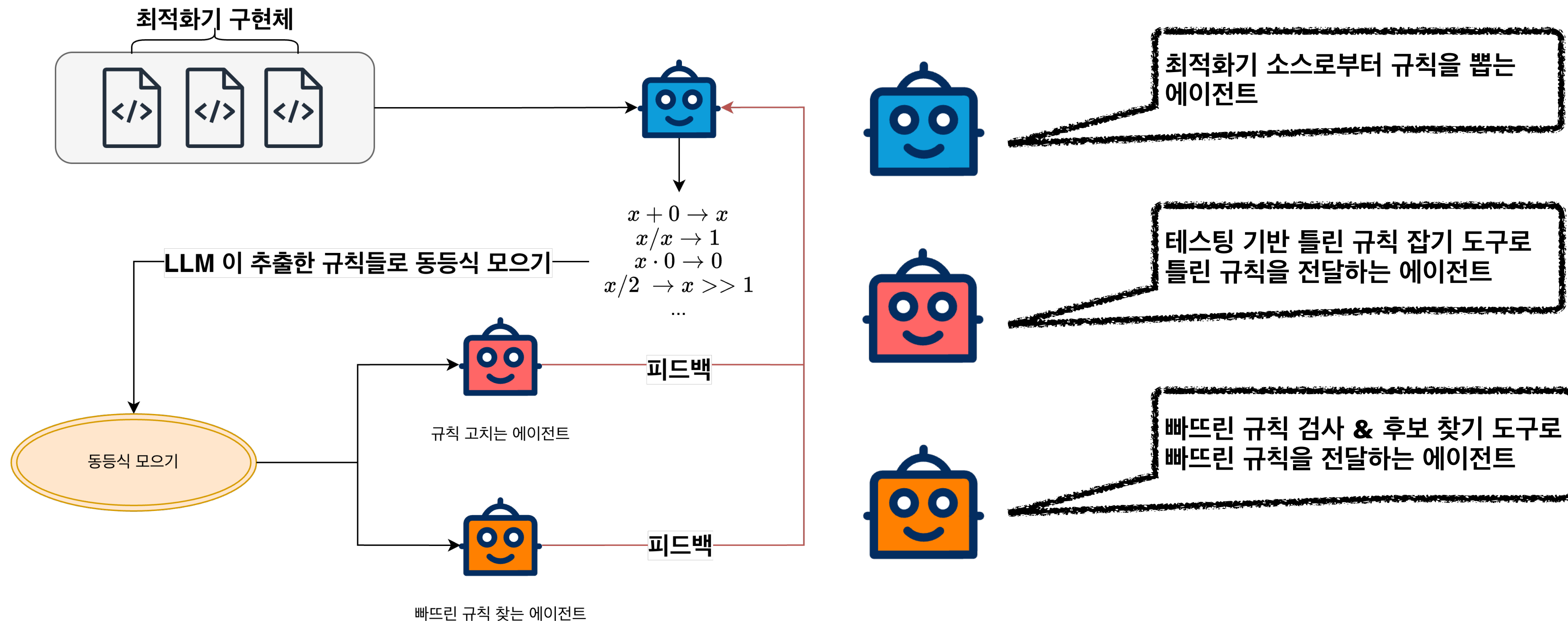
- 기존 최적화기 코드는 단순한 변환 규칙만으로 이루어져 있지 않다
- 여러 조건과 검사 과정이 함께 섞여 있어, 무엇을 어떤걸로 바꾸는지 명확하게 안드러남
 - **LLM** 을 활용하여 방대한 양의 최적화기 코드를 다시쓰기 규칙으로 변환하자

LLM 이 주도하여 기존 최적화기를 동등식 모으기 기반 최적화기로 만들기



LLM이 주도하는 전체 그림

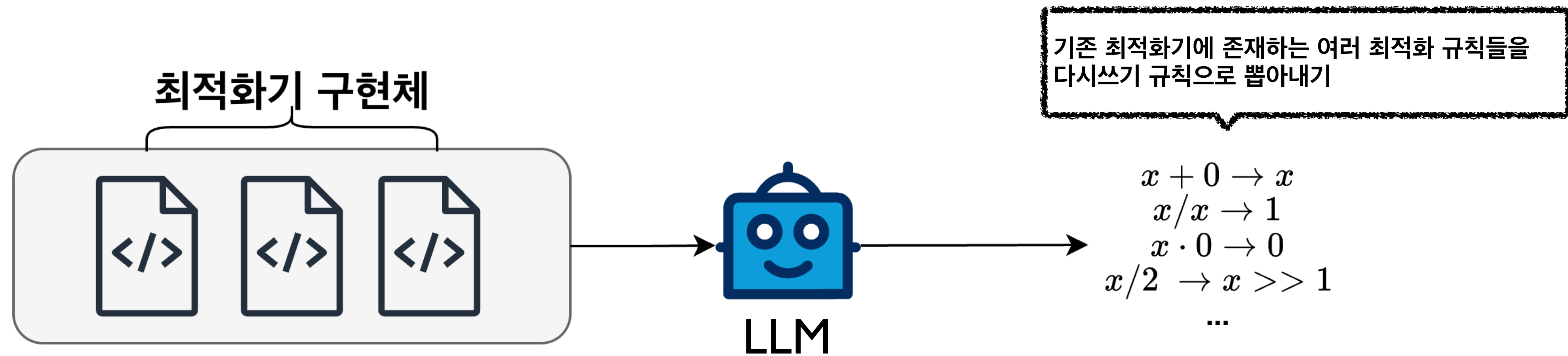
현재는 에이전트들이 피드백을 주고 받으며 자동으로 최적화기를 생성하는 것이 아닌 사용자가 수동으로 도구를 사용하여 그 결과를 LLM에게 직접 전달하는 식이다



- **LLM 에이전트를 다수 굴리면서**

- 기존 컴파일러 구현체로부터 ‘자동으로’ 동등식 모으기 기반 최적화기를 생성하고 싶다
- 모든 것을 믿고 맡기는 대신, 에이전트들에게 도움이 되는 도구를 쥐어주면서

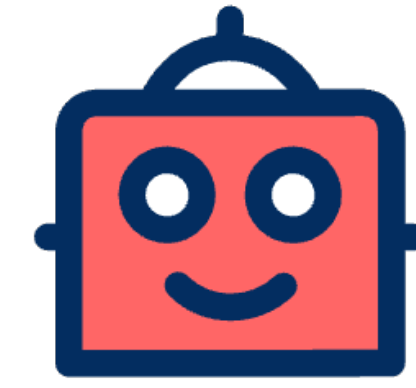
컴파일러 구현체로부터 다시쓰기 규칙 뽑기



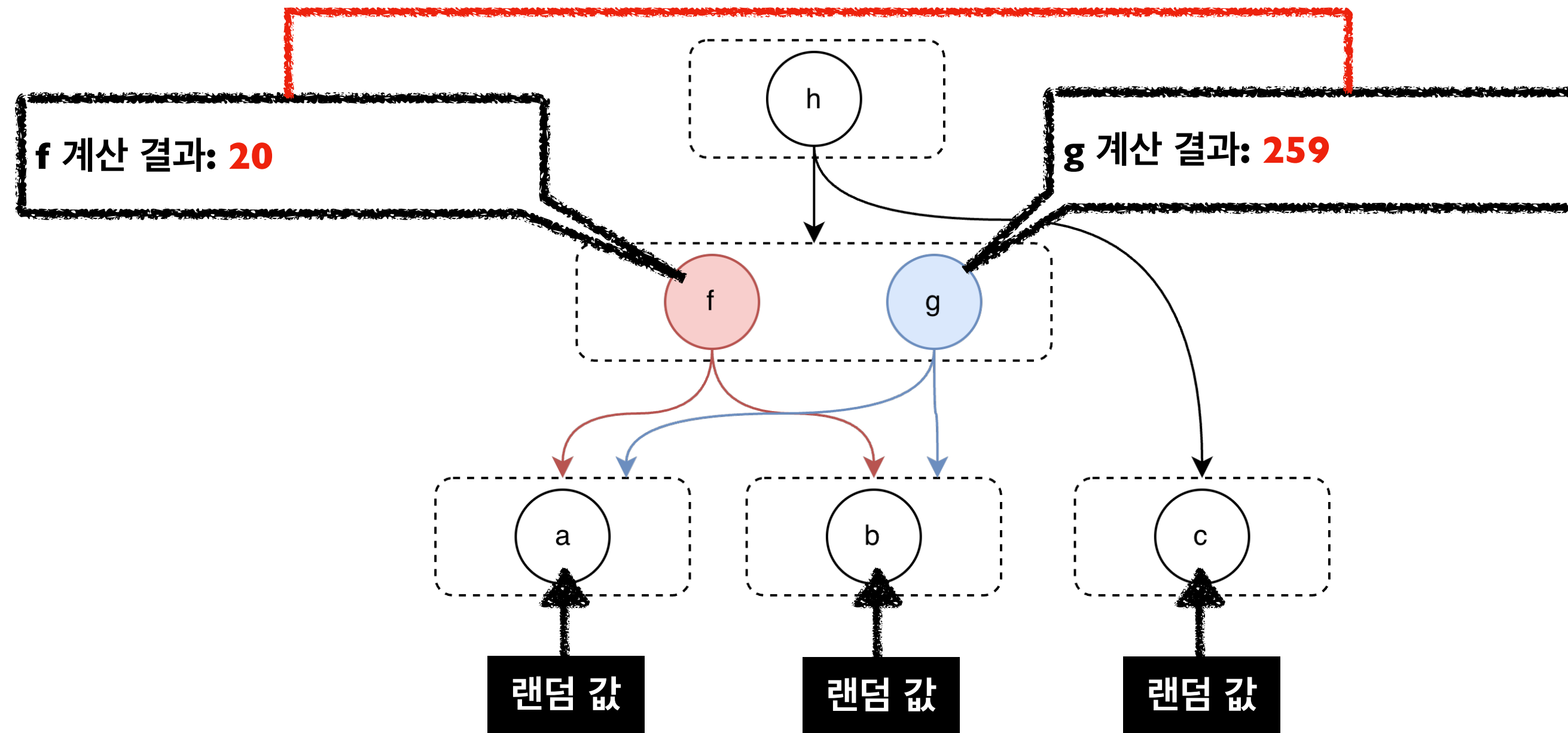
- 생각보다 잘 뽑아내더라

- 다만, 뽑아낸 다시쓰기 규칙이 정말 올바른지 & 빠뜨린 규칙은 없는지는 불명확하다
- 이녀석만 믿어서는 안된다

반복 테스트 기반 틀린 규칙 잡기



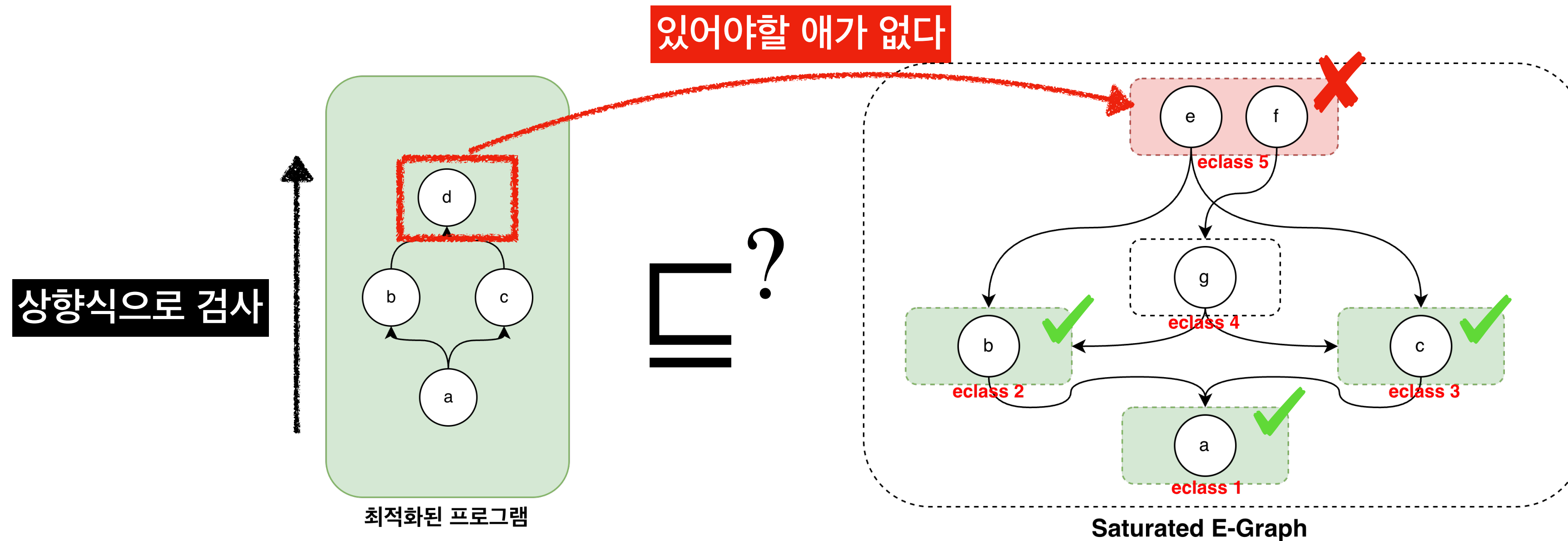
불일치! $\implies$ f와 g가 동등하다고 여긴 다시쓰기 규칙에 버그가 있다는 것



- **LLM** 이 구현한 다시쓰기 규칙을 온전히 신뢰하긴 어렵다
- **LLM**이 다시쓰기 규칙을 잘 구현했다 $\implies$ **E-Graph** 상에서 동등식들의 계산 결과가 같아야한다
- 만약 일치하지 않는다면, 그것들을 동등하다고 여긴 다시쓰기 규칙이 틀렸다

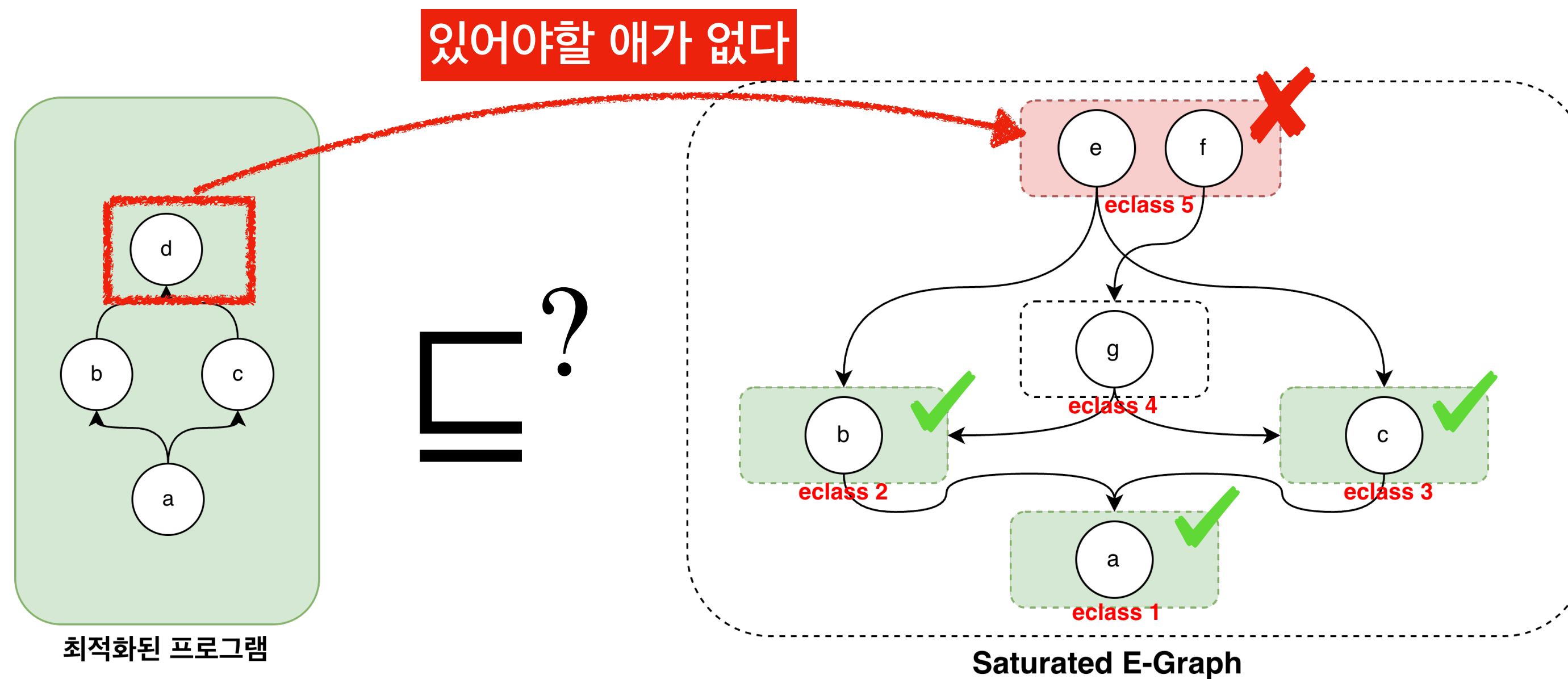
빠뜨린 규칙이 있는지 검사

기존 최적화기가 만들 수 있는 프로그램이 E-Graph 에 있는지



- **LLM** 이 컴파일러 구현체에서 미처 뽑아내지 못한 규칙이 있을 수 있다
- 충분히 규칙을 뽑아내어 동등식이 모두 모인 **E-Graph** 를 만들었다면
 - 기존 최적화기가 만든 프로그램 (식) 또한 그 **E-Graph** 에 있을 것이다

빠뜨린 규칙이 무엇인지 후보 나열하기 🤖

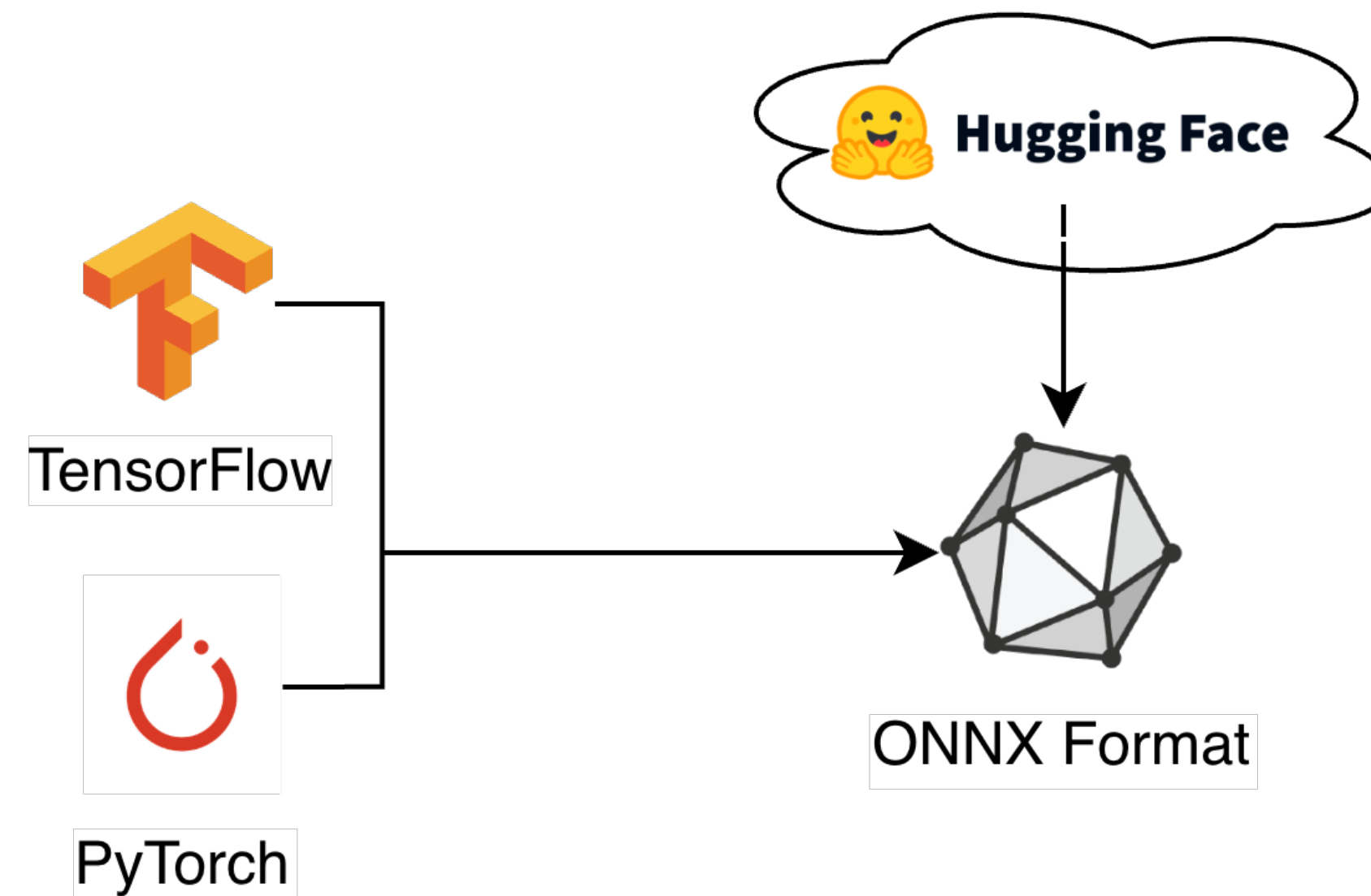


- **b, c** 를 재료로 갖는 **d** 가 **E-Graph** 에 없다 (원래라면 **eclass 5** 동등식 모임에 있어야)
- **eclass 5** 에는 **d** 가 아닌 **e, f** 가 존재한다
- **LLM** 이 빠뜨린 규칙은 ??? $\rightarrow d(b, c)$ 임은 분명하다
 - **e, f** 는 있으므로 빠뜨린 규칙 후보는 $e(b, c) \rightarrow d(b, c) \mid f(g(b, c)) \rightarrow d(b, c)$

적용

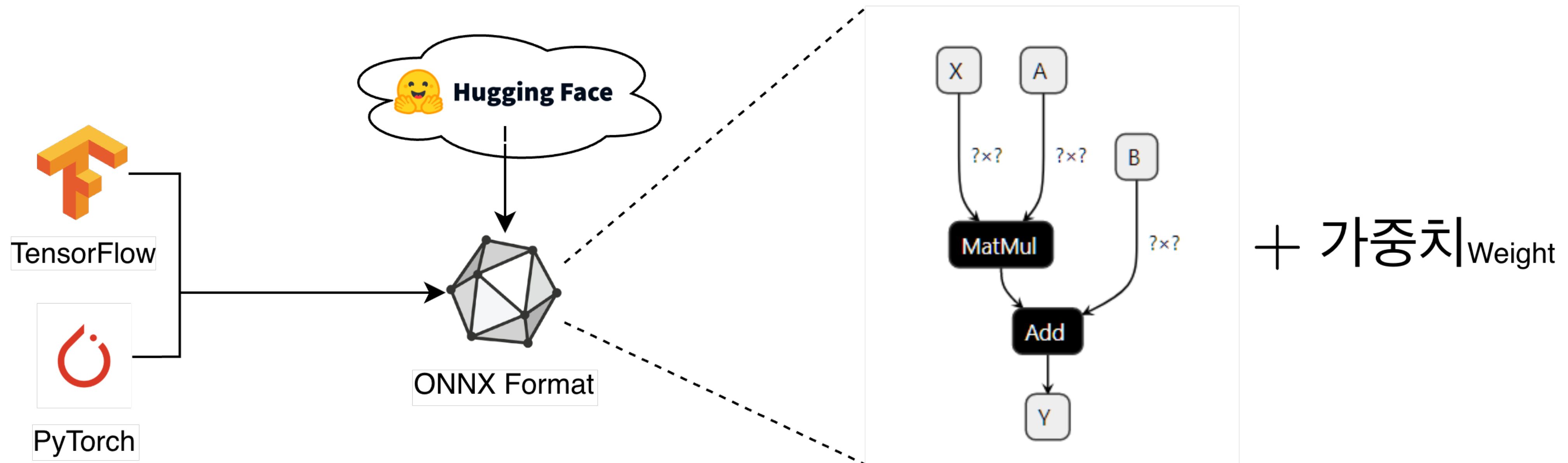
ONNX 모델 & ONNX 모델 최적화기

ONNX ?



- **ONNX** 는 AI 모델을 표현하는 표준 “**포맷**” ($\simeq$ pdf)
 - 많은 딥러닝 프레임워크가 구현한 모델을 **ONNX** 포맷으로 내보낼 수 있다
 - 다양한 문서 작성 도구가 “pdf로 추출을” 지원하듯이

ONNX 모델 구조



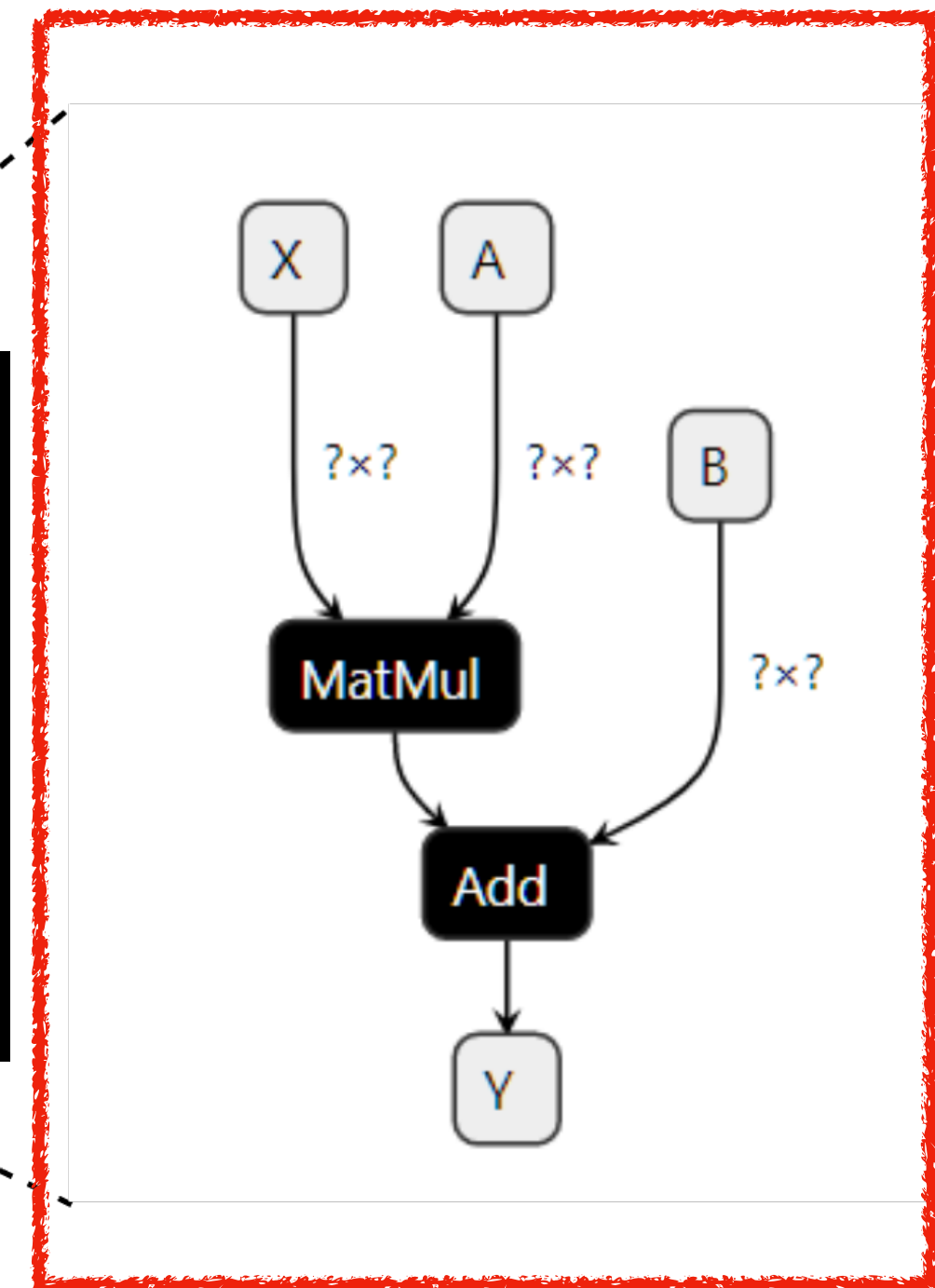
- **ONNX** 는 뉴럴넷을 (연산 그래프 , 미리 학습된 가중치 값) 으로 표현한다
- 연산 그래프란, 뉴럴넷으로 들어가는 입력이 어떤 연산을 통해서 어떻게 흘러가는지를 표현

목표



“더 빠른 추론”이 가능한
연산 그래프를 만드는 최적화기
(우리 방법론을 이용해서 만든)

PyTorch



연산 그래프

+ 가중치_{Weight}

실험 결과

추론 속도 비교

- ONNXRUNTIME 최적화기 (baseline)
- ONNXRUNTIME-TF (트랜스포머 전용 최적화기)
- E-ONNX (Ours, 동등식 모으기 기반 최적화기)

- 대부분의 모델에서 기존 최적화기보다 더 빠르거나 같은 수준의 연산 그래프를 내놓았다
- 결과로 나온 연산 그래프는 **GPU** 에서 실행하거나 **CPU** 에서 실행하거나 더 빠르거나 같은 수준이다

최대 58% 빠름 (GPU)
최대 49% 빠름 (CPU)
- 하드웨어 밀접한 저수준의 최적화 방법 없이 고수준의 연산 그래프 레벨에서 행한 최적화만 적용해도 효과가 뛰어났다

Model	GPU (A5000)			CPU		
	ORT	ORT-TF	E-ONNX	ORT	ORT-TF	E-ONNX
<i>Encoder (BERT-family, batch=1, seq=128)</i>						
bert_tiny	0.0	+5.7	+58.1	0.0	+35.5	+35.5
minilm-l6	0.0	+4.2	+56.8	0.0	+27.1	+27.3
distil_bert	0.0	+3.7	+46.7	0.0	+8.2	+9.3
embeddinggemma-300m	0.0	–	+39.7	0.0	–	+8.0
albert-base	0.0	+7.3	+29.4	0.0	+6.1	+9.3
unixcoder-base	0.0	+7.5	+28.3	0.0	+7.9	+7.7
bge-m3	0.0	+7.8	+28.2	0.0	+2.8	+2.4
granite-embedding	0.0	–	+27.9	0.0	–	+10.0
roberta-base	0.0	+6.7	+24.6	0.0	+5.7	+5.3
electra-base	0.0	+25.0	+24.6	0.0	+6.6	+6.6
deberta-v3-base	0.0	–	+22.2	0.0	–	+4.9
qwen3-embedding-0.6b	0.0	–2.7	+13.0	0.0	–5.5	+7.2
mobilebert	0.0	–	+11.0	0.0	–	+7.1
Average	0.0	+7.2	+31.6	0.0	+10.5	+10.8
<i>Decoder (LLM, batch=4, seq=1, past=1024)</i>						
qwen-0.5b	0.0	–	+39.2	0.0	–	+39.3
gpt2_mini	0.0	+10.3	+39.0	0.0	+1.5	+43.5
gpt2-openai	0.0	+3.8	+35.7	0.0	+0.5	+49.0
granite-4.1-8b	0.0	–	+32.5	0.0	–	+24.9
llama-3.2-1b-fp16	0.0	–	+32.4	0.0	–	+35.7
mistral-7b-fp16	0.0	–	+28.5	0.0	–	+28.7
qwen-7b	0.0	–	+28.4	0.0	–	+26.9
codellama-7b-python	0.0	–	+14.2	0.0	–	+13.0
llama2-7b	0.0	–	+14.1	0.0	–	+13.2
gemma-3-270m	0.0	–	+9.2	0.0	–	+3.2
gemma-2b-fp16	0.0	–	+3.6	0.0	–	+2.7
Average	0.0	+7.1	+25.2	0.0	+1.0	+25.5
<i>Vision-Transformer (ViT/Swin, batch=4)</i>						
swin-base	0.0	–11.6	+22.8	0.0	–0.9	+1.2
swin-large	0.0	–7.6	+19.5	0.0	–0.4	+0.7
clip-vit-large	0.0	+1.3	+8.8	0.0	–0.2	+1.5
deit-base	0.0	+1.6	+5.2	0.0	+0.8	+2.7
vit	0.0	+1.4	+5.1	0.0	+0.3	+2.8
vit-large	0.0	+1.8	+0.4	0.0	+0.9	+4.7
Average	0.0	–2.2	+10.3	0.0	+0.1	+2.3